UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

ACTIVIDAD 3. BÚSQUEDA LOCAL: HILL CLIMBING Y SIMULATED ANNELING

Asignatura: Fundamentos de Inteligencia Artificial

Docente: Mtro. Javier García Lara

Carrera: Ingeniería en Tecnologías de la Información e Innovación Digital

Grupo: 171 **Cuatrimestre:** 7

Alumno: Suarez Melonie Magdalena

1. Introducción

Los algoritmos de búsqueda local se utilizan para encontrar soluciones a problemas de optimización. En lugar de revisar todas las soluciones posibles, estos algoritmos parten de una solución inicial y buscan mejorarla mediante cambios sucesivos.

En esta actividad se implementan dos algoritmos: Hill Climbing y Simulated Annealing. Ambos se utilizan para minimizar una función matemática, pero trabajan de manera diferente. Hill Climbing selecciona movimientos que mejoran la solución actual, mientras que Simulated Annealing también puede aceptar temporalmente movimientos que empeoran el resultado para explorar otras posibilidades.

2. Planteamiento del problema

Se desea encontrar el valor de x que minimice la siguiente función:

$$f(x) = (x-3)^2 + 2$$

El valor de \(x\) debe encontrarse entre -10 y 10 . El objetivo es encontrar el menor costo posible y comparar los resultados de los dos algoritmos.

El mínimo teórico de esta función se encuentra en $x = 3$, donde $f(x) = 2$. Este valor servirá como referencia para evaluar el desempeño de los algoritmos.

3. Modelo del problema

Elemento	Descripción
Estado	Un valor numérico de x
Estado inicial	$x = -8.$
Acciones	Cambiar el valor de x mediante pasos pequeños o aleatorios.

Función objetivo	$f(x) = (x-3)^2 + 2$
Meta	Encontrar el menor valor posible de la función.
Restricción	$-10 \leq x \leq 10$
Costo	El resultado de evaluar la función; cuanto menor sea, mejor.

4. Descripción de los algoritmos

4.1 Hill Climbing

Hill Climbing, o escalada de colinas, comienza con una solución inicial y revisa sus vecinos. Si encuentra una solución con un costo menor, se mueve hacia ella. El procedimiento continúa hasta que ya no encuentra mejoras o llega al límite de iteraciones.

Su ventaja es que resulta sencillo de implementar y puede encontrar buenas soluciones rápidamente. Sin embargo, en problemas más complejos puede quedarse atrapado en un mínimo local.

4.2 Simulated Annealing

Simulated Annealing, o recocido simulado, también busca minimizar una función, pero utiliza cambios aleatorios. Acepta los movimientos que mejoran la solución y, en determinadas condiciones, acepta movimientos que empeoran temporalmente el resultado.

La probabilidad de aceptar un movimiento desfavorable depende de la temperatura. Al principio la temperatura es alta y permite explorar más; después disminuye gradualmente, haciendo que el algoritmo sea más selectivo.

5. Código Python

```python
import random
import math
import matplotlib.pyplot as plt

# Funcion que queremos minimizar
def funcion(x):
    return (x - 3) ** 2 + 2

# Algoritmo Hill Climbing
def hill_climbing(x, paso=0.1, max_iter=500):
    historial = [funcion(x)]

    for _ in range(max_iter):
        vecinos = [
            max(-10, min(10, x - paso)),
            max(-10, min(10, x + paso))
        ]

        mejor = min(vecinos, key=funcion)

        if funcion(mejor) >= funcion(x):
            break

        x = mejor
        historial.append(funcion(x))

    return x, funcion(x), historial

# Algoritmo Simulated Annealing
def simulated_annealing(
    x, temperatura=10.0, enfriamiento=0.97,
    temperatura_min=0.001, max_iter=500
):
    mejor_x = x
    historial = [funcion(x)]
    T = temperatura

    for _ in range(max_iter):
        candidato = max(
            -10, min(10, x + random.uniform(-1, 1))
        )

        diferencia = funcion(candidato) - funcion(x)

        # Acepta mejoras y algunos movimientos peores
        if diferencia < 0 or random.random() < math.exp(-diferencia / T):
            x = candidato

        if funcion(x) < funcion(mejor_x):
            mejor_x = x
```

```
        historial.append(funcion(mejor_x))
        T *= enfriamiento

        if T < temperatura_min:
            break

    return mejor_x, funcion(mejor_x), historial

# Programa principal
if __name__ == "__main__":
    random.seed(42)

    x1, costo1, historial1 = hill_climbing(-8)
    x2, costo2, historial2 = simulated_annealing(-8)

    print("RESULTADOS DE HILL CLIMBING")
    print(f"x encontrado: {x1:.4f}")
    print(f"Costo: {costo1:.4f}")

    print("\nRESULTADOS DE SIMULATED ANNEALING")
    print(f"x encontrado: {x2:.4f}")
    print(f"Costo: {costo2:.4f}")

    print("\nMinimo teorico: x = 3, f(x) = 2")

    plt.plot(historial1, label="Hill Climbing")
    plt.plot(historial2, label="Simulated Annealing")
    plt.xlabel("Iteraciones")
    plt.ylabel("Mejor costo encontrado")
    plt.title("Comparacion de convergencia")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("convergencia_actividad3.png")
    plt.show()
```

6. Procedimiento de implementación

1. Se define la función matemática que se desea minimizar.
2. Se establece el intervalo permitido para los valores de x.
3. Se implementa Hill Climbing para probar movimientos cercanos y aceptar mejoras.
4. Se implementa Simulated Annealing para explorar movimientos aleatorios utilizando una temperatura decreciente.

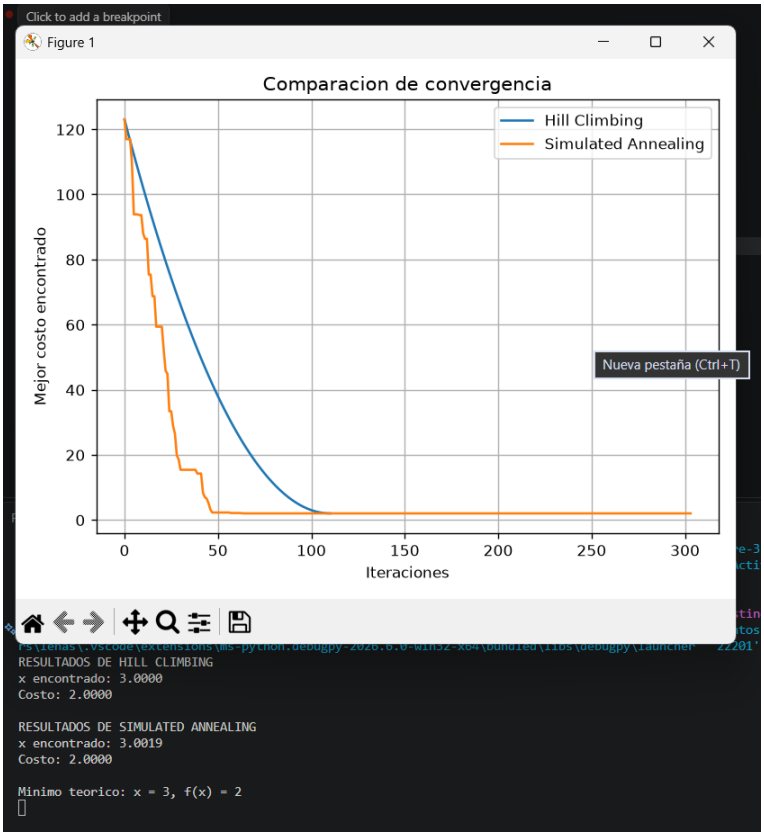
5. Se ejecutan ambos algoritmos desde el mismo estado inicial.
6. Se comparan los costos encontrados y se genera una gráfica de convergencia.

7. Resultados y análisis

El programa muestra el valor de x encontrado y el costo final de cada algoritmo. También genera una gráfica para visualizar cómo disminuye el mejor costo durante las iteraciones.

El resultado ideal es alcanzar $x = 3$ y un costo de 2. Hill Climbing puede llegar directamente a ese mínimo en esta función porque tiene una forma convexa y un único mínimo. Simulated Annealing realiza una búsqueda aleatoria, por lo que su resultado depende de los movimientos que acepte durante la ejecución.

Para completar esta sección, se deben registrar los valores que aparezcan al ejecutar el programa. No es recomendable inventar tiempos ni resultados experimentales.



8. Conclusión

En esta actividad se observó que Hill Climbing y Simulated Annealing son técnicas útiles para resolver problemas de optimización. Hill Climbing es sencillo y aprovecha las mejoras inmediatas, mientras que Simulated Annealing permite explorar otras soluciones al aceptar temporalmente algunos movimientos desfavorables.

La principal diferencia es que el segundo algoritmo tiene mayor capacidad de exploración, aunque sus resultados dependen de sus parámetros. Ambos pueden ser útiles según las características del problema que se desea resolver.

9. Declaración de uso de inteligencia artificial

Se utilizó ChatGPT como apoyo para organizar la explicación del tema y elaborar una estructura inicial del código.

10. Referencias

Georgiou, I. (2026). SUNSPOT PREDICTION WITH a NEURAL NETWORK: AN APPLICATION OF ADVANCED ARTIFICIAL INTELLIGENCE IN SECONDARY EDUCATION. *International Journal Of Global Education (IJGE)*. http://www.ijge.net/index.php/php/article/view/84

Nylund, K., Mankoff, J., & Potluri, V. (2025). MatplotAlt: A Python Library for Adding Alt Text to Matplotlib Figures in Computational Notebooks. *Computer Graphics Forum*, *44*(3). https://doi.org/10.1111/cgf.70119

Tan, J., Feitosa, D., Avgeriou, P., & Lungu, M. (2020). Evolution of technical debt remediation in Python: A case study on the Apache Software Ecosystem. *Journal Of Software Evolution And Process*, *33*(4). https://doi.org/10.1002/smr.2319